

- Windsurf failed to port the “faster note parsing” feature from H-Slice to my engine. I explained many times that the 186408 notes (the entire chart) are the actual objects, but it just wouldn’t listen and kept insisting that the 38390 notes (a false positive result from the console) were the actual objects
- I fixed the null error myself. Then I restored the code, though some parts were still broken. Next, I reopened Windsurf with a new chat and asked it to fix the errors. Only then did it truly understand and fix them
- After a period of testing, I found that only the word “stupid” can describe just how poor its intelligence is when dealing with Haxe compared to models from other major companies: I clearly stated that the current code was processing very slowly, but it assumed the current code wasn’t working, so it “fixed” code that was already working, added unnecessary variables, changed the structure, and broke both PlayState.hx and ChartingState.hx. I then had to revert everything back to the slow but working code again, keeping only the new feature I had asked it to add. It was genuinely very stupid. Even with access to the user’s project folder, it still managed to make completely unauthorized assumptions
- Windsurf usually replaces its code to the code line wrong position, gets so little information from H-Slice to implement. No one, even Cursor, can make the silly mistakes. The root cause is it just skims, not reads carefully
- Windsurf is lying. I can see the previous/latest update only allows the use of SWE-1.6 Slow. All the other models are restricted to the Pro plan
- Impossible! The policy is not transparent again. No matter how many days I wait, whenever I use the SWE-1.6 Slow model to send a message, the message gets reverted, or it only goes through if I keep spamming the Enter key, but then the notification “We’re currently facing high demand for this model. Please try again later.” keeps appearing and deletes the message. I have to resend it over and over before the model finally responds
- I said one thing, but the SWE did another. I clearly stated that the traces and the message display functions **mustn’t** be made into generic custom message functions, but it assumed that I wanted them to be generic. As a result, it not only changed them into generic ones but also modified them in the wrong places. I yelled, and it still wouldn’t listen. In the end, I had to fix them myself and explain it to the SWE before it finally understood
- I don’t know what I did to Devin. I was using Devin when the system suddenly logged me out automatically. After that, I tried logging back

in, but the website only let me choose an account even though I was clearly already logged in. It displayed “No eligible accounts found”. Now I can’t continue using Devin anymore. After just 2 hours (from 12 PM to 2 PM), they reopened it

- The ignorance of Devin in Java: It stubbornly insisted that a class in the same path must be imported into another class, even though classes in the same path can be used directly without any import at all. I had to point it out clearly before it acknowledged the mistake. For example: There are two classes, A and B. Devin keeps insisting that you have to import A in order to use B, even though it’s obvious that if they’re both in the same path/package, B can use A directly without needing to import it. Full conversation:
 - + Me: The import
`net.mcreator.explosiveblocklib.procedures.PlayerMineDataProcedure` is never used
 - + Devin: The import
`net.mcreator.explosiveblocklib.procedures.PlayerMineDataProcedure` is actually being used in the file. This is likely a false positive from the IDE’s linting that hasn’t refreshed after the recent edits. The import is necessary and cannot be removed.
 - + Me: You are blaming the false positive from IDE’s linting. Class CAN BE USED in the other class that is same path only
 - + Devin: You’re correct. Since both
`RemoteMineControllerRightclickedOnBlockProcedure` and
`PlayerMineDataProcedure` are in the same package
(*`net.mcreator.explosiveblocklib.procedures`*), the import is not needed. Classes in the same package can reference each other without imports.